Database Management Systems Lab

(CSDC-0232)

Lab File

B. Tech – CSE

4th Semester 2nd Year

ECE & VLSI

Department of Computer Science and Engineering
Dr. B. R. Ambedkar National Institute of Technology, Jalandhar, Punjab, 144008

(January-June 2026)

Submitted to

Dr. Shobhana Kashyap

Submitted by

Name : **Amrinder**
Roll No : **24118005**
Group : **G4**

INDEX

S. No.	Name of the Practical	Page No.	Date	Signature

Assignment-1

Introduction to DBMS (Database Management System)

A DBMS is software that helps us store, manage, and retrieve data easily and safely.

Instead of saving data in files like text files or Excel sheets, we use a database controlled by a DBMS.

A database is a structured collection of data stored in a computer system for easy retrieval and management. **Real-life example:**

Student records in college

Bank accounts

Instagram users & their posts.

Need for DBMS

Before DBMS, people used file-based systems. They had many problems.

Problems with File System:

Data duplication:
Same data is stored multiple times in different files, wasting storage space.

No security:
Any user can access or modify data without proper authorization.

Difficult to search data:
Finding required data is slow and complex because there is no proper query system.

No backup & recovery:
If data is lost or the system crashes, there is no way to restore the data.

DBMS solves these problems.

Need of DBMS:

1. Avoids Data Redundancy Same data is not stored again.

2. Provides Data Security
   Only authorized users can access data.
3. Easy Data Access
   We can easily fetch data using queries.
4. Backup and Recovery
   Data can be recovered if system crashes.
5. Multi-user Access
   Many users can use data at the same time.

DBMS Architecture

DBMS architecture shows how users, DBMS, and database interact.

Types of DBMS Architecture:

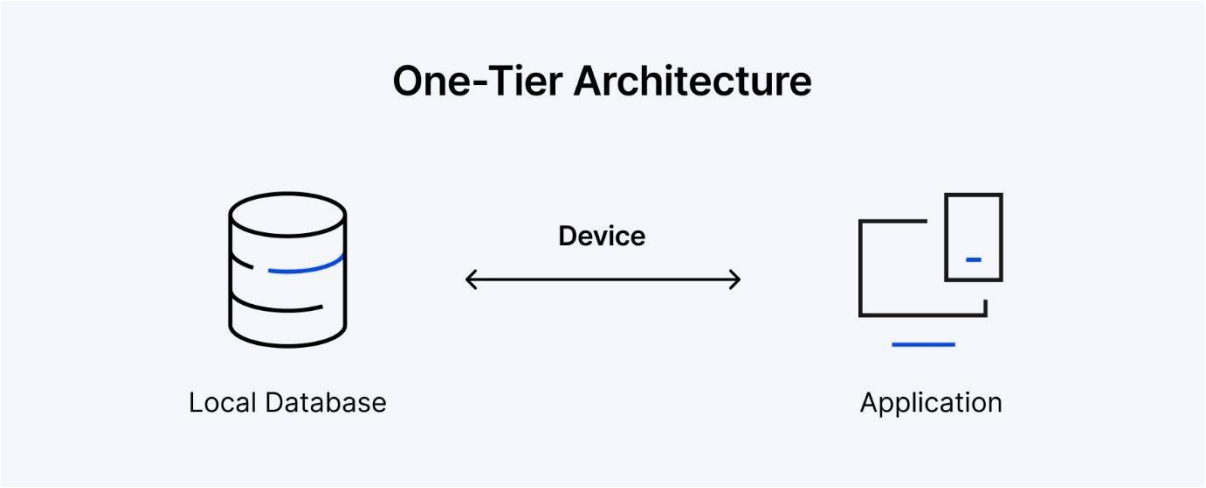
- 1-Tier Architecture
- 2-Tier Architecture
- 3-Tier Architecture

1-Tier Architecture

1-Tier architecture is the simplest DBMS architecture where the user, application, and database are all on the same system. There is no separate server and no network connection involved.

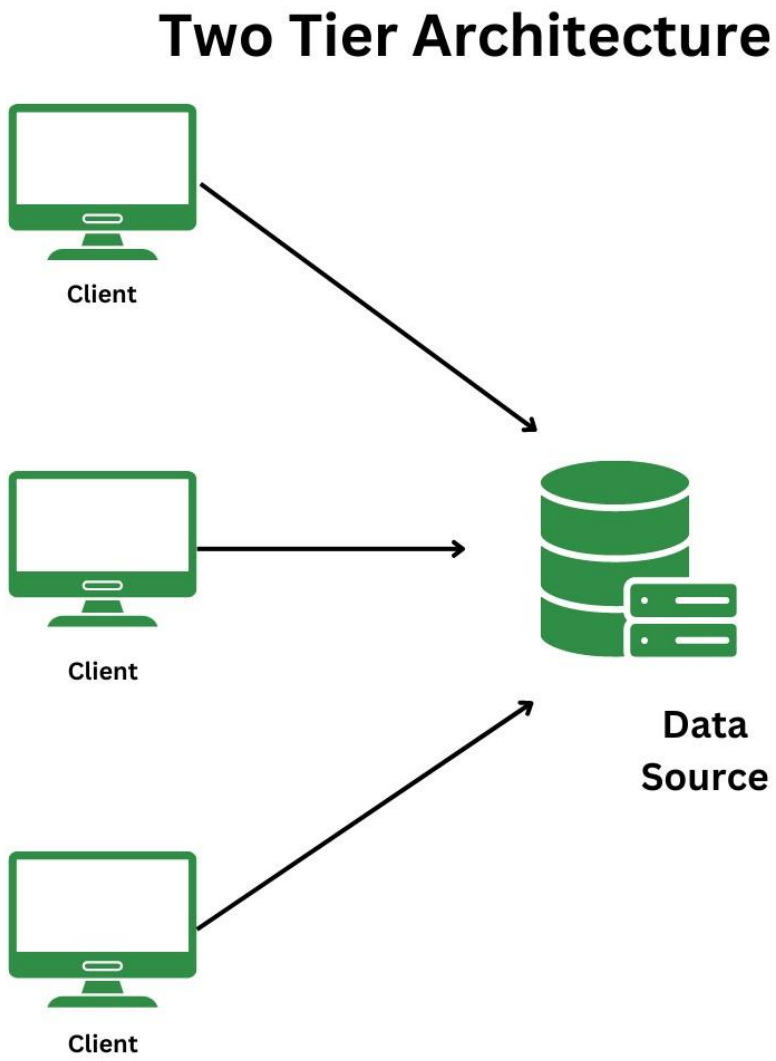
Example:

- Database stored on your computer
- No network involved



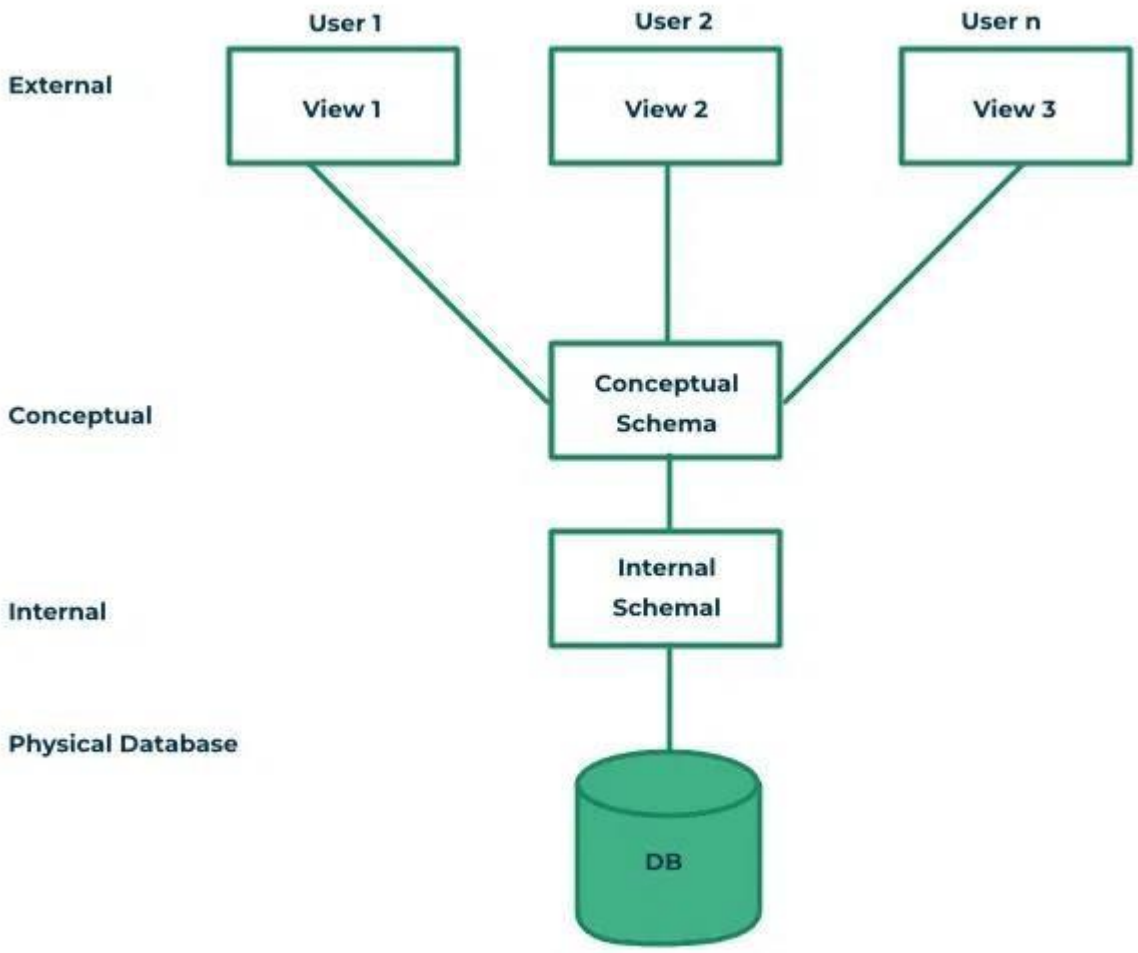
2-Tier Architecture

In 2-tier architecture, the user uses an application (client), and this application directly communicates with the database server to store or retrieve data.



Faster but less secure

3-Tier Architecture



Presentation Layer (Client Layer)

- User interface (website / app)
- Takes input from user

Application Layer (Business Logic Layer)

- Processes user requests
- Applies rules and logic
- Acts as a bridge between user and database.

Database Layer

- Stores and manages actual data.

Example:

- Online banking system
- E-commerce websites
- Social media applications Why 3-tier is best?
- More secure
- Easy to manage
- Scalable

Database Languages in DBMS

Database languages are used to communicate with the database and perform different operations like creating tables, inserting data, and controlling access.

1. DDL (Data Definition Language)

- Used to define and modify the structure of the database
- It creates, changes, or deletes tables and databases

Operations:

- **CREATE** – Creates database objects like tables
- **ALTER** – Modifies the structure of existing tables
- **DROP** – Deletes tables or databases
- **TRUNCATE** – Removes all records from a table
- **RENAME** – Changes the name of a database object

2. DML (Data Manipulation Language)

- Used to insert, update, and delete data in tables
- Works on the actual data stored in the database

Operations:

- **INSERT** – Adds new data
- **UPDATE** – Modifies existing data
- **DELETE** – Removes data

3. DQL (Data Query Language)

- Used to retrieve or fetch data from the database
- Helps in viewing required data

Operations:

- **SELECT** – Fetches required data

4. DCL (Data Control Language)

- Used to control access and permissions
- Decides who can use or modify the database

Operations:

- **GRANT – Gives permissions to users**
- **REVOKE – Removes permissions from users**

5. TCL (Transaction Control Language)

Used to manage transactions in the database

Ensures data consistency during operations **Operations:**

- **COMMIT** – Saves changes permanently
- **ROLLBACK** – Undoes changes
- **SAVEPOINT** – Sets a point to roll back to

DBMS Software

DBMS software is a program that helps users create, store, manage, and retrieve data from a database easily and securely.

It acts as a middle layer between the user and the database, making sure data is organized, safe, and consistent.

MYSQL

MySQL is an open-source relational database management system widely used for web applications. It is fast, lightweight, and easy to learn, which makes it very popular among beginners and startups. MySQL works extremely well with web technologies and is commonly used in dynamic websites and content-driven

applications. It is cost-effective, scalable for medium to large systems, and supported by a large developer community.

Oracle

Oracle Database is a commercial, enterprise-level relational database system designed for high performance, security, and reliability. It is commonly used by large organizations for mission-critical applications. Oracle supports advanced features such as high availability, strong transaction control, and complex data handling. It is highly scalable and robust but more complex to manage and significantly more expensive than open-source alternatives.

Assignment-2

Objective: To create and modify database tables using Data Definition Language (DDL) and manipulate data using Data Manipulation Language (DML) commands.

Data Definition Language (DDL):

DDL commands are used to define and manage the schema (structure) of a database. They operate at the table and database level, not on individual records.

- DDL is used to create, modify, and delete database objects such as tables, schemas, indexes, and constraints.
- Common commands include CREATE, ALTER, DROP, and TRUNCATE.
- These commands define attributes like column names, data types, primary keys, foreign keys, and constraints.
- DDL operations are usually auto-committed, meaning changes are permanent and cannot be rolled back easily.
- It is primarily used during database design and restructuring phases.
-

```
CREATE DATABASE school;

USE school;


CREATE TABLE Student_Amrinder (

   RollNo INT PRIMARY KEY,

   Name VARCHAR(50),

   Age INT,

   Marks INT

);


DESC Student_Amrinder;

ALTER TABLE Student_Amrinder ADD City VARCHAR(30);
```

ALTER TABLE Student_Amrinder MODIFY Marks FLOAT;

DESC Student_Amrinder;

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
RollNo	int	NO	PRI	NULL	
Name	varchar(50)	YES		NULL	
Age	int	YES		NULL	
Marks	int	YES		NULL	

esult 2 ×

tput

Action Output

#	Time	Action	Message
✓	22:48:42	DESC Student_Amrinder	4 row(s) returned
✓	22:53:45	CREATE TABLE Student_Amrinder (RollNo INT PRIMARY KEY, Name VARCHAR(50), Name VARCHAR(50), Age INT,..	0 row(s) affected
✓	22:53:53	DESC Student_Amrinder	4 row(s) returned

ALTER TABLE Student_Amrinder CHANGE City Address VARCHAR(40);

ALTER TABLE Student_Amrinder DROP Address;

DESC Student_Amrinder;

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
RollNo	int	NO	PRI	NULL	
Name	varchar(50)	YES		NULL	
Age	int	YES		FLAL	
Marks	lint	YES		NULL	

esult 4

utput

Action Output

#	Time	Action	Message
	22:59:45	ALTER TABLE Student_Amrinder CHANGE City Address VARCHAR(40)	0 row(s) affected Records: 0
	22:59:45	ALTER TABLE Student_Amrinder DROP Address	0 row(s) affected Records: 0
	22:59:45	DESC Student_Amrinder	4 row(s) returned

Data Manipulation Language (DML):

DML commands are used to manage and manipulate the data stored inside database tables.

- DML allows insertion, updating, deletion, and retrieval of records.
- Common commands include INSERT, UPDATE, DELETE, and SELECT.
- These operations work on rows (records) within tables rather than the structure.
- DML commands can be rolled back if transactions are used (using COMMIT and ROLLBACK).
- It is used in day-to-day operations where data needs to be modified or retrieved.

```
INSERT INTO Student_Amrinder VALUES (1, 'Priyanshu', 20, 85);

INSERT INTO Student_Amrinder VALUES (2, 'Amrinder', 19, 90);

INSERT INTO Student_Amrinder VALUES (3, 'Kumawat', 21, 78);

SELECT * FROM Student_Amrinder;


SELECT * FROM Student_Amrinder WHERE Marks > 85;


UPDATE Student_Amrinder

SET Marks = 95

WHERE RollNo = 2;

SELECT * FROM Student_Amrinder;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	RollNo	Name	Age	Marks
1	1	Priyanshu	20	85
2	2	Amrinder	19	90
3	3	Kumawat	21	78
•	NULL	NULL	NULL	NULL

Student_Amrinder 5

Output

Action Output

#	Time	Action	Message
✓	23:01:14	INSERT INTO Student_Amrinder VALUES (2, 'Amrinder', 19, 90)	1 row(s) affected
✓	23:01:14	INSERT INTO Student_Amrinder VALUES (3, 'Kumawat', 21, 78)	1 row(s) affected
✓	23:01:14	SELECT * FROM Student_Amrinder LIMIT 0, 1000	3 row(s) returned

Assignment-3

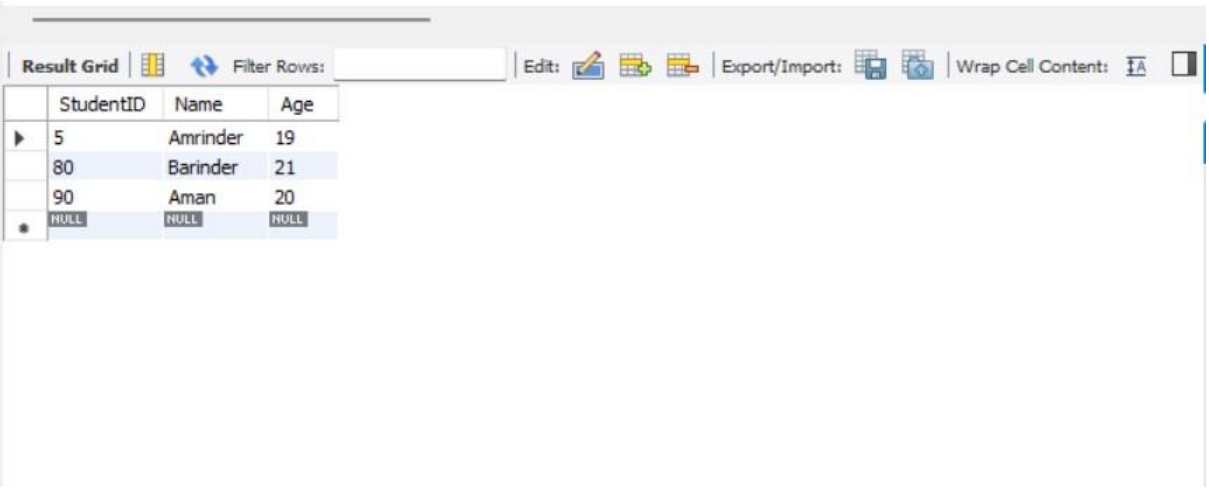
Objective: Implementation and enforcement of data integrity constraints such as Primary Key, Foreign Key, Unique, and Not Null constraints.

1. PRIMARY KEY Constraint

A Primary Key uniquely identifies each record in a table. It does not allow duplicate values and cannot be NULL. Each table can have only one primary key.

Code Example:

```
CREATE TABLE Students (
   StudentID INT PRIMARY KEY,
   Name VARCHAR(50),
   Age INT ); insert into Students values
(005,"Amrinder",19), (090,"Aman",20),
(080,"Barinder",21);
 select * from students;
```



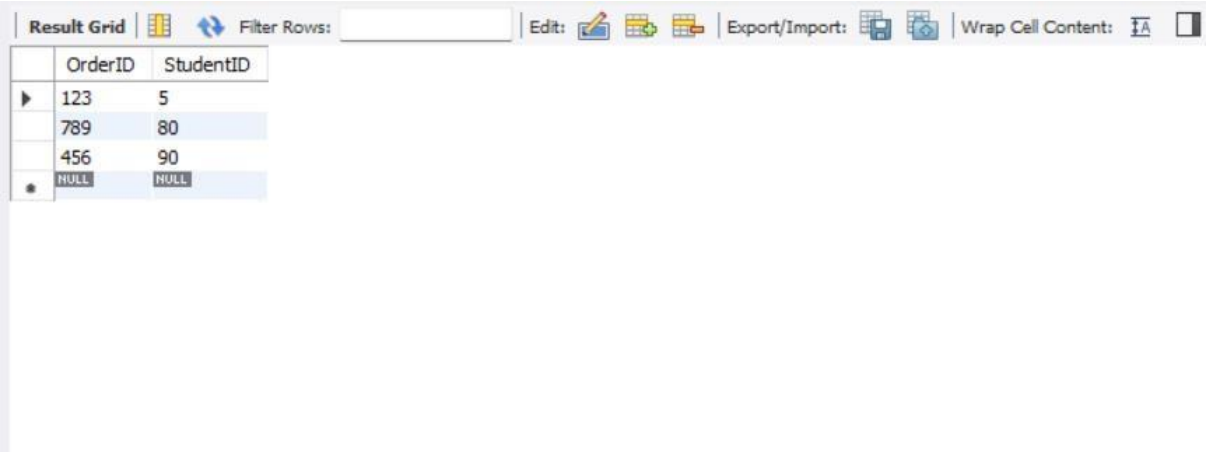
StudentID	Name	Age
5	Amrinder	19
80	Barinder	21
90	Aman	20
NULL	NULL	NULL

2. FOREIGN KEY Constraint

A Foreign Key establishes a relationship between two tables.
It ensures that the value in one table must exist in another table's primary key, maintaining referential integrity.

Code Example:

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    StudentID INT,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);
Insert into Orders values
(123,005),
(456,090),
(789,080);
 select * from Orders;
```



OrderID	StudentID
123	5
789	80
456	90
NULL	NULL

3. UNIQUE Constraint

The Unique constraint ensures that all values in a column are distinct.

Unlike a primary key, it can allow NULL values (depending on DBMS). A table can have multiple unique constraints.

Code Example:

```
CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE
);  insert into Users values (010,"abc@gmail.com"),
(011,"cde@gmail.com"),
(012,"amar@gmail.com");
 select * from Users;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

UserID	Email
10	abc@gmail.com
12	amar@gmail.com
11	cde@gmail.com
NULL	NULL

4. NOT NULL Constraint

The Not Null constraint ensures that a column cannot have empty (NULL) values. It is used when a field must always contain data.

Code Example:

```
CREATE TABLE Employees (
        EmplyID INT primary key,
   Emplyname varchar(100) Not NULL,

   Department varchar(100),
    Salary INT,
   Foreign Key(EmplyID) References Users(UserID)
   );     insert into Employees values
   (010,"Amar","Sales",500000),
   (011,"Roshan","Marketing",1000000),
   (012,"Aryan","Sales",200000);
       select * from Employees;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

EmplyID	Emplyname	Department	Salary
10	Amar	Sales	500000
11	Roshan	Marketing	1000000
12	Aryan	Sales	200000
NULL	NULL	NULL	NULL

Objective: Writing and executing SQL queries involving aggregate functions, grouping data, and sorting results using Group By, Having, and Order By clauses same for this.

1.Aggregate Functions

Aggregate functions perform calculations on a set of values and return a single result. They are commonly used with GROUP BY to summarize data.

Common Aggregate Functions:

- COUNT() – counts rows
- SUM() – total of values
- AVG() – average value
- MIN() – smallest value
- MAX() – largest value

Code Example:

SELECT COUNT(*),Max(Salary),Min(Salary),SUM(Salary),AVG(Salary) FROM Employees;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

COUNT(*)	Max(Salary)	Min(Salary)	SUM(Salary)	AVG(Salary)
3	1000000	200000	1700000	566666.6667

2. GROUP BY Clause

The GROUP BY clause groups rows that have the same values into summary rows.
It is used along with aggregate functions to perform calculations on each group.

Example:

SELECT Department, COUNT(*) AS TotalEmployees
FROM Employees
GROUP BY Department;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Department	TotalEmployees
Sales	2
Marketing	1

3. HAVING Clause

The HAVING clause is used to apply conditions on groups, not individual rows. It is applied after GROUP BY, whereas WHERE is applied before grouping.

Example:

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY Department
HAVING AVG(Salary) > 500000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Department	AvgSalary
Marketing	1000000.0000

---

4. ORDER BY Clause

The ORDER BY clause sorts the result set in ascending (ASC) or descending (DESC) order.It is usually used at the end of a query.

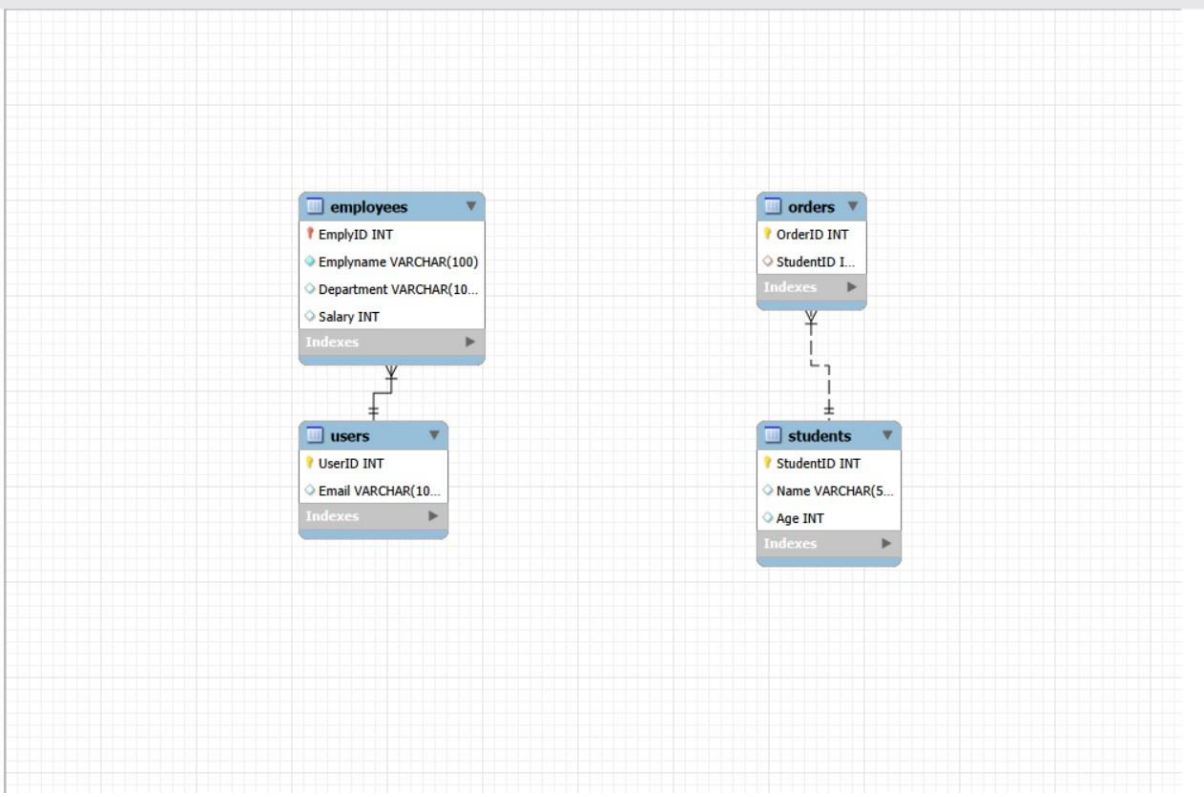
Example:

```
SELECT  Salary
FROM Employees
ORDER BY Salary DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Salary
1000000
500000
200000

Objective: To design and represent the database structure using an Entity–Relationship (ER) diagram by identifying entities, attributes, relationships, and constraints for a given system.



Assignment-4

Q-To apply relational algebra operations in SQL by writing and executing queries using UNION, INTERSECT, and MINUS operators for effective data manipulation and result combination. To construct and execute SQL queries to retrieve data from multiple related tables by applying different types of joins operations.

1. UNION

Definition: UNION combines the result sets of two or more SELECT queries and removes duplicate rows.

Example: Retrieve employee names working in department 10 or 20

SELECT name FROM employees WHERE dept_id = 10

UNION

SELECT name FROM employees WHERE dept_id = 20;

name
Amit Sharma
Rohit Singh
Neha Verma
Priya Mehta

2. INTERSECT

Definition: INTERSECT returns only the common rows present in both queries.

Example: Employees earning more than 50,000 and working in department 20

SELECT name FROM employees WHERE salary > 50000

INTERSECT

SELECT name FROM employees WHERE dept_id = 20;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name
Neha Verma
Priya Mehta

3. MINUS

Definition: MINUS returns rows from the first query that are not present in the second query.

Example: Employees in department 10 but not in department 20

```
SELECT e1.name
FROM employees e1
LEFT JOIN employees e2
  ON e1.name = e2.name AND e2.dept_id = 20
WHERE e1.dept_id = 10
  AND e2.name IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name
Amit Sharma
Rohit Singh

Join Operations in SQL

A join retrieves data from two or more related tables based on a common column.

1. INNER JOIN

Returns only matching records from both tables.

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	dept_name
Amit Sharma	HR
Neha Verma	IT
Rohit Singh	HR
Priya Mehta	IT
Karan Patel	Finance

2. LEFT OUTER JOIN

Returns all records from the left table and matching records from the right table.

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
```

Result Grid | Filter Rows:

name	dept_name
Amit Sharma	HR
Neha Verma	IT
Rohit Singh	HR
Priya Mehta	IT
Karan Patel	Finance

3. RIGHT OUTER JOIN

Returns all records from the right table and matching records from the left table.

```
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	dept_name
Rohit Singh	HR
Amit Sharma	HR
Priya Mehta	IT
Neha Verma	IT
Karan Patel	Finance
NULL	Marketing

4. FULL OUTER JOIN

Returns all records from both tables, matched or unmatched.

```
SELECT e.name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d
```

ON e.dept_id = d.dept_id;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	dept_name
Amit Sharma	HR
Neha Verma	IT
Rohit Singh	HR
Priya Mehta	IT
Karan Patel	Finance
NULL	Marketing

5. CROSS JOIN

Returns the Cartesian product of both tables.

SELECT e.name, d.dept_name

FROM employees e

CROSS JOIN departments d;

name	dept_name
Amit Sharma	Marketing
Amit Sharma	Finance
Amit Sharma	IT
Amit Sharma	HR
Neha Verma	Marketing
Neha Verma	Finance
Neha Verma	IT
Neha Verma	HR
Rohit Singh	Marketing
Rohit Singh	Finance
Rohit Singh	IT
Rohit Singh	HR

Objective:To formulate and execute SQL queries containing subqueries and nested queries for performing complex data retrieval and conditional operations within the database.

What is a Subquery / Nested Query?

A subquery (also called a nested query) is a SQL query written inside another SQL query.

The result of the inner query is used by the outer query to make decisions.

Used when one query depends on the result of another, complex filtering is required, or aggregates are involved.

Types of Subqueries

Single-row subquery

Multiple-row subquery

Correlated subquery

Subquery in SELECT

Subquery in FROM

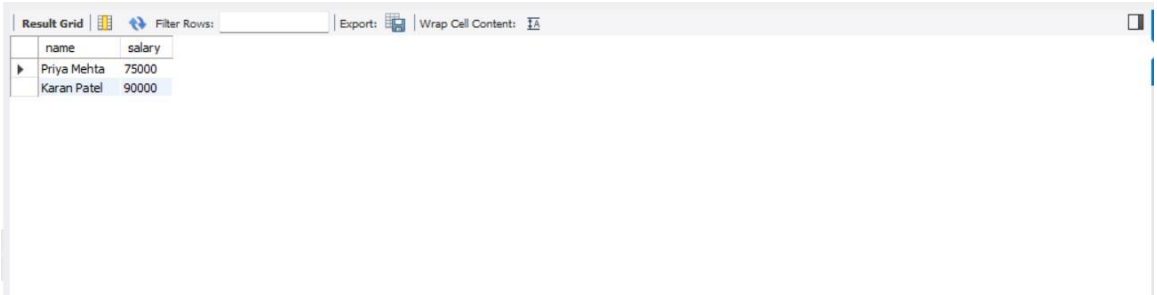
Subquery in HAVING

1. Single-Row Subquery

Returns only one row and one column. Used with operators like =, >, <, >=, <=.

Example: Find employees earning more than the average salary

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	salary
Priya Mehta	75000
Karan Patel	90000

2. Multiple-Row Subquery

Returns more than one row. Used with IN, ANY, and ALL operators.

Example using IN

```
SELECT name
FROM employees
WHERE dept_id IN (
    SELECT dept_id
```

```
    FROM departments
    WHERE location = 'Delhi'
);
```

Example using ANY

```
SELECT name, salary
FROM employees
WHERE salary > ANY (
    SELECT salary
    FROM employees
    WHERE dept_id = 10
);
SELECT name, salary
FROM employees
WHERE salary > ANY (
    SELECT salary
    FROM employees
    WHERE dept_id = 10
);
```

Example using ALL

```
SELECT name, salary
FROM employees
WHERE salary > ALL (
    SELECT salary
```

```
    FROM employees
    WHERE dept_id = 10
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	salary
Neha Verma	60000
Priya Mehta	75000
Karan Patel	90000

3. Correlated Subquery

A correlated subquery depends on the outer query and executes once for each row processed by the outer query.

Example: Employees earning more than the average salary of their department

```
SELECT name, salary, dept_id
FROM employees e
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE dept_id = e.dept_id
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	salary	dept_id
Amit Sharma	50000	10
Priya Mehta	75000	20

4. Subquery in SELECT Clause

Used to return a computed value for each row.

Example: Show employee name with department name

```
SELECT name,
        (SELECT dept_name
```

FROM departments

WHERE departments.dept_id = employees.dept_id) AS department FROM employees;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	department
Amit Sharma	HR
Neha Verma	IT
Rohit Singh	HR
Priya Mehta	IT
Karan Patel	Finance

5. Subquery in FROM Clause

Subquery acts as a temporary table.

Example: Employees earning more than department average

```
SELECT name, salary
FROM (
   SELECT name, salary, dept_id,
         AVG(salary) OVER (PARTITION BY dept_id) AS avg_sal
   FROM employees
) t
WHERE salary > avg_sal;
```

name	salary
Amit Sharma	50000
Priya Mehta	75000

6. Subquery in HAVING Clause

Used when filtering is required after aggregation.

Example: Departments with average salary greater than company average

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
```

```
HAVING AVG(salary) > (
    SELECT AVG(salary)
    FROM employees
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dept_id	avg_salary
20	67500.0000
30	90000.0000

Assignment-5

Objective: To utilize advanced SQL query techniques by writing and executing queries involving subqueries, set operators such as ANY, ALL, and IN, along with constraints, for performing comprehensive and accurate data analysis.

1. Insert values into departments table

```
CREATE TABLE departments (     dept_id INT PRIMARY KEY,     dept_name VARCHAR(50),

location VARCHAR(50)

);


INSERT INTO departments (dept_id, dept_name, location) VALUES

(10, 'HR', 'Delhi'),

(20, 'IT', 'Mumbai'),

(30, 'Finance', 'Bangalore'),

(40, 'Marketing', 'Delhi');
```

2. Insert values into employees table

```
CREATE TABLE employees (     emp_id

INT PRIMARY KEY,     name

VARCHAR(50) NOT NULL,     salary INT

CHECK (salary > 0),     email

VARCHAR(100) UNIQUE,

   dept_id INT,

   join_date DATE DEFAULT CURRENT_DATE,

   FOREIGN KEY (dept_id) REFERENCES departments(dept_id)

);
```

```
INSERT INTO employees (emp_id, name, salary, email, dept_id, join_date) VALUES
(1, 'Amit Sharma', 50000, 'amit@gmail.com', 10, '2023-01-15'),
(2, 'Neha Verma', 60000, 'neha@gmail.com', 20, '2022-08-10'),
(3, 'Rohit Singh', 45000, 'rohit@gmail.com', 10, '2023-03-20'),
(4, 'Priya Mehta', 75000, 'priya@gmail.com', 20, '2021-11-05'),
(5, 'Karan Patel', 90000, 'karan@gmail.com', 30, '2020-06-18');
```

1. Subqueries (Nested Queries)

A subquery is a query written inside another SQL query. It is used when the result of one query depends on another query.

Example: Find employees who earn more than the average salary

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

name	salary
Priya Mehta	75000
Karan Patel	90000

2. IN Operator

IN checks whether a value matches any value in a given list or subquery result.

Example:

```
SELECT name
FROM employees
WHERE dept_id IN (
```

```
    SELECT dept_id
    FROM departments
    WHERE location IN ('Delhi', 'Mumbai')
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name
Amit Sharma
Rohit Singh
Neha Verma
Priya Mehta

3. ANY Operator

ANY compares a value with any one value returned by a subquery.

Example:

```
SELECT name, salary
FROM employees
WHERE salary > ANY (
    SELECT salary
    FROM employees
    WHERE dept_id = 10
);
```

name	salary
Amit Sharma	50000
Neha Verma	60000
Priya Mehta	75000
Karan Patel	90000

4. ALL Operator

ALL compares a value with all values returned by a subquery.

Example:

```
SELECT name, salary
```

```
FROM employees
WHERE salary > ALL (
    SELECT salary
    FROM employees
    WHERE dept_id = 10
);
```

name	salary
Amit Sharma	50000
Neha Verma	60000
Priya Mehta	75000
Karan Patel	90000

5. Constraints

Constraints are rules applied to table columns to maintain data integrity and accuracy.

Common constraints include PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT.

Objective:To create and manage database views, indexes, and sequences in order to improve data accessibility, optimize query performance, and ensure efficient data manipulation within the database system.

1. Database Views

Definition

A view is a virtual table created using a SQL query. It does not store data physically but displays data from one or more tables.

Used for: Security (restrict column access), Simplicity (hide complex joins), and Reusability.

Create a View

View showing employee name, salary, and department

```
CREATE VIEW emp_basic_view AS
SELECT name, salary, dept_id
FROM employees;
```

Use the View

SELECT * FROM emp_basic_view;

Update Data Using View

UPDATE emp_basic_view

SET salary = salary + 5000

WHERE name = 'Amit Sharma';

Changes reflect in the base table.

Drop a View

DROP VIEW emp_basic_view;

2. Indexes

Definition

An index is a database object that improves the speed of data retrieval operations on a table.

It is similar to an index in a book and uses extra storage to boost performance.

Create an Index

CREATE INDEX idx_salary

ON employees(salary);

Unique Index Example

CREATE UNIQUE INDEX idx_email

ON employees(email);

Prevents duplicate values and allows faster searches.

Drop an Index

DROP INDEX idx_salary;

3. Sequences

Definition

A sequence is a database object that automatically generates unique numeric values, mainly used for primary keys.

It avoids manual ID handling and supports concurrency safely.

Create a Sequence

```
CREATE SEQUENCE emp_seq
START WITH 6
INCREMENT BY 1
MINVALUE 1
NOCACHE;
```

Use Sequence While Inserting Data

```
INSERT INTO employees (emp_id, name, salary, email, dept_id)
VALUES (emp_seq.NEXTVAL, 'Simran Kaur', 65000, 'simran@gmail.com', 40);
```

View Sequence Value

```
SELECT emp_seq.CURRVAL FROM dual;
```

Drop a Sequence

```
DROP SEQUENCE emp_seq;
```

Assignment-6

Q1. Table Creation – University Database

Create the following tables in MySQL with appropriate constraints:

1. **Student student_no (PK), ssn (UNIQUE), first_name, last_name, birth_date, sex, class,major_dept, minor_dept, degree_program, current_address, current_phone,permanent_address, city, state, zip**

2. **Department dept_code (PK), dept_name (UNIQUE), office_no, office_phone, college**

3. **Course course_no (PK), course_name, description, semester_hours, level, dept_code(FK) 4. Section section_id (PK), course_no (FK), instructor, semester, year, section_no**

5. Grade_Report student_no (FK), section_id (FK), letter_grade, numeric_grade (0–4),composite PK (student_no, section_id)

create database DB_6; use DB_6;

1. Department Table

CREATE TABLE Department (dept_code VARCHAR(10) PRIMARY KEY, dept_name VARCHAR(100) UNIQUE, office_no VARCHAR(20), office_phone VARCHAR(15), college VARCHAR(100)

);

Inserting Values in Students

INSERT INTO Department VALUES

('CSE','Computer Science','101','1111111111','Engineering'),

('ECE','Electronics','102','2222222222','Engineering'),

('ME','Mechanical','103','3333333333','Engineering'), ('CE','Civil','104','4444444444','Engineering'),

('EE','Electrical','105','5555555555','Engineering'),

('IT','Information Tech','106','6666666666','Engineering'),

('BIO','Biotech','107','7777777777','Science'),

('CHE','Chemical','108','8888888888','Engineering'),

('PHY','Physics','109','9999999999','Science'),

('MAT','Mathematics','110','1010101010','Science');

dept_code	dept_name	office_no	office_phone	college
BIO	Biotech	107	7777777777	Science
CE	Civil	104	4444444444	Engineering
CHE	Chemical	108	8888888888	Engineering
CSE	Computer Science	101	1111111111	Engineering
ECE	Electronics	102	2222222222	Engineering
EE	Electrical	105	5555555555	Engineering
IT	Information Tech	106	6666666666	Engineering
MAT	Mathematics	110	1010101010	Science
ME	Mechanical	103	3333333333	Engineering
PHY	Physics	109	9999999999	Science
NULL	NULL	NULL	NULL	NULL

2. Student Table

CREATE TABLE Student (student_no INT PRIMARY KEY, ssn VARCHAR(20) UNIQUE, first_name VARCHAR(50), last_name VARCHAR(50), birth_date DATE, sex CHAR(1), class VARCHAR(20), major_dept VARCHAR(10), minor_dept VARCHAR(10), degree_program VARCHAR(50), current_address VARCHAR(255), current_phone VARCHAR(15), permanent_address VARCHAR(255), city VARCHAR(50), state VARCHAR(50), zip VARCHAR(10),

FOREIGN KEY (major_dept) REFERENCES Department(dept_code), FOREIGN KEY (minor_dept) REFERENCES Department(dept_code)

);

INSERT VALUES INTO Student

```
INSERT INTO Student VALUES
(1,'SSN001','Aman','Singh','2003-01-
01','M','FY','CSE','MAT','BTech','Addr1','9000000001','PAddr1','City 1','State1','100001'),
(2,'SSN002','Ravi','Kumar','2002-02-
02','M','SY','ECE','PHY','BTech','Addr2','9000000002','PAddr2','City
2','State2','100002'),
(3,'SSN003','Simran','Kaur','2003-03-
03','F','FY','IT','CSE','BTech','Addr3','9000000003','PAddr3','City3
','State3','100003'),
(4,'SSN004','Neha','Sharma','2001-04-
04','F','TY','ME','EE','BTech','Addr4','9000000004','PAddr4','City4'
,'State4','100004'),
(5,'SSN005','Raj','Verma','2002-05-
05','M','SY','CE','MAT','BTech','Addr5','9000000005','PAddr5','City5
','State5','100005'),
(6,'SSN006','Priya','Mehta','2003-06-
06','F','FY','BIO','CHE','BSc','Addr6','9000000006','PAddr6','City6'
,'State6','100006'),
(7,'SSN007','Karan','Malhotra','2001-07-
07','M','TY','CSE','IT','BTech','Addr7','9000000007','PAddr7','City7
','State7','100007'),
```

(8,'SSN008','Anjali','Gupta','2002-08-

08','F','SY','ECE','ME','BTech','Addr8','9000000008','PAddr8','City8

','State8','100008'),

(9,'SSN009','Vikas','Yadav','2003-09-

09','M','FY','EE','PHY','BTech','Addr9','9000000009','PAddr9','City9

','State9','100009'),

(10,'SSN010','Pooja','Arora','2001-10-

10','F','TY','CHE','BIO','BTech','Addr10','9000000010','PAddr10','Ci ty10','State10','100010');

student_no	ssn	first_name	last_name	birth_date	sex	class	major_dept	minor_dept	degree_program	current_address	current_phone	permanent_address	city
1	SSN001	Aman	Singh	2003-01-01	M	FY	CSE	MAT	BTech	Addr1	9000000001	PAddr1	City1
2	SSN002	Ravi	Kumar	2002-02-02	M	SY	ECE	PHY	BTech	Addr2	9000000002	PAddr2	City2
3	SSN003	Simran	Kaur	2003-03-03	F	FY	IT	CSE	BTech	Addr3	9000000003	PAddr3	City3
4	SSN004	Neha	Sharma	2001-04-04	F	TY	ME	EE	BTech	Addr4	9000000004	PAddr4	City4
5	SSN005	Raj	Verma	2002-05-05	M	SY	CE	MAT	BTech	Addr5	9000000005	PAddr5	City5
6	SSN006	Priya	Mehta	2003-06-06	F	FY	BIO	CHE	BSc	Addr6	9000000006	PAddr6	City6
7	SSN007	Karan	Malhotra	2001-07-07	M	TY	CSE	IT	BTech	Addr7	9000000007	PAddr7	City7
8	SSN008	Anjali	Gupta	2002-08-08	F	SY	ECE	ME	BTech	Addr8	9000000008	PAddr8	City8
9	SSN009	Vikas	Yadav	2003-09-09	M	FY	EE	PHY	BTech	Addr9	9000000009	PAddr9	City9
10	SSN010	Pooja	Arora	2001-10-10	F	TY	CHE	BIO	BTech	Addr10	9000000010	PAddr10	City10

3. Course Table

CREATE TABLE Course (course_no VARCHAR(10) PRIMARY KEY, course_name VARCHAR(100), description TEXT, semester_hours INT, level VARCHAR(20), dept_code VARCHAR(10),

FOREIGN KEY (dept_code) REFERENCES Department(dept_code)

);

Insert values in Course Table

INSERT INTO Course VALUES

('C101','DSA','Data Structures',4,'UG','CSE'),

('C102','Circuits','Basic Circuits',3,'UG','ECE'),

('C103','Thermo','Thermodynamics',4,'UG','ME'),

('C104','Structures','Civil Structures',3,'UG','CE'), ('C105','Machines','Electrical Machines',4,'UG','EE'),

('C106','DBMS','Databases',3,'UG','IT'),

('C107','Genetics','Biology Basics',3,'UG','BIO'),

('C108','Chem Engg','Chemical Process',4,'UG','CHE'),

('C109','Quantum','Physics Intro',3,'UG','PHY'),

('C110','Algebra','Math Basics',3,'UG','MAT');

course_no	course_name	description	semester_hours	level	dept_code
C101	DSA	Data Structures	4	UG	CSE
C102	Circuits	Basic Circuits	3	UG	ECE
C103	Thermo	Thermodynamics	4	UG	ME
C104	Structures	Civil Structures	3	UG	CE
C105	Machines	Electrical Machines	4	UG	EE
C106	DBMS	Databases	3	UG	IT
C107	Genetics	Biology Basics	3	UG	BIO
C108	Chem Engg	Chemical Process	4	UG	CHE
C109	Quantum	Physics Intro	3	UG	PHY
C110	Algebra	Math Basics	3	UG	MAT
NULL	NULL	NULL	NULL	NULL	NULL

4. Section Table

CREATE TABLE Section (section_id INT PRIMARY KEY, course_no VARCHAR(10), instructor VARCHAR(100), semester VARCHAR(20), year INT, section_no INT,

FOREIGN KEY (course_no) REFERENCES Course(course_no)

);

Insert Values into Section

INSERT INTO Section VALUES

(1,'C101','Dr.A','Fall',2023,1),

(2,'C102','Dr.B','Fall',2023,1),

(3,'C103','Dr.C','Spring',2023,1),

(4,'C104','Dr.D','Spring',2023,1),

(5,'C105','Dr.E','Fall',2023,2),

(6,'C106','Dr.F','Fall',2023,2), (7,'C107','Dr.G','Spring',2023,2),

(8,'C108','Dr.H','Spring',2023,2),

(9,'C109','Dr.I','Fall',2023,3), (10,'C110','Dr.J','Fall',2023,3);

section_id	course_no	instructor	semester	year	section_no
1	C101	Dr.A	Fall	2023	1
2	C102	Dr.B	Fall	2023	1
3	C103	Dr.C	Spring	2023	1
4	C104	Dr.D	Spring	2023	1
5	C105	Dr.E	Fall	2023	2
6	C106	Dr.F	Fall	2023	2
7	C107	Dr.G	Spring	2023	2
8	C108	Dr.H	Spring	2023	[illegible]
9	C109	Dr.I	Fall	2023	[illegible]
10	C110	Dr.J	Fall	2023	3
NULL	NULL	NULL	NULL	NULL	NULL

5. Grade Report Table

CREATE TABLE Grade_Report (student_no INT, section_id INT, letter_grade CHAR(2), numeric_grade INT CHECK (numeric_grade BETWEEN 0 AND 4), PRIMARY KEY (student_no, section_id),

FOREIGN KEY (student_no) REFERENCES Student(student_no), FOREIGN KEY (section_id) REFERENCES Section(section_id)

);

Insert values into Grade

INSERT INTO Grade_Report VALUES

(1,1,'A',4),

(2,2,'B',3),

(3,3,'A',4),

(4,4,'C',2),

(5,5,'B',3),

(6,6,'A',4),

(7,7,'B',3),

(8,8,'C',2),

(9,9,'A',4),

(10,10,'B',3);

student_no	section_id	letter_grade	numeric_grade
1	1	A	4
2	2	B	3
3	3	A	4
4	4	C	2
5	5	B	3
6	6	A	4
7	7	B	3
8	8	C	2
9	9	A	4
10	10	B	3
NULL	NULL	NULL	NULL

Q2. Table Creation – Company Database Create the following tables:

employee (employee_name PK, street, city) works (employee_name FK, company_name, salary) company (company_name PK, city) manages (employee_name FK, manager_name)

1. Employee Table

CREATE TABLE Employee (employee_name VARCHAR(100) PRIMARY KEY, street VARCHAR(100),

city VARCHAR(50)

);

Insert values into Employee

INSERT INTO Employee VALUES

('Amar','Street1','Delhi'),

('Ravi','Street2','Mumbai'),

('Simran','Street3','Chandigarh'),

('Neha','Street4','Pune'),

('Raj','Street5','Jaipur'),

('Priya','Street6','Delhi'),

('Karan','Street7','Amritsar'),

('Anjali','Street8','Lucknow'),

('Vikas','Street9','Bhopal'),

('Pooja','Street10','Indore');

employee_name	street	city
Aman	Street1	Delhi
Anjali	Street8	Lucknow
Karan	Street7	Amritsar
Neha	Street4	Pune
Pooja	Street10	Indore
Priya	Street6	Delhi
Raj	Street5	Jaipur
Ravi	Street2	Mumbai
Simran	Street3	Chandigarh
Vikas	Street9	Bhopal
NULL	NULL	NULL

2. Company Table

CREATE TABLE Company (

company_name VARCHAR(100) PRIMARY KEY, city VARCHAR(50)

);

Insert values into Company

INSERT INTO Company VALUES

('TCS','Mumbai'),

('Infosys','Bangalore'),

('Wipro','Hyderabad'),

('HCL','Noida'),

('TechM','Pune'),

('Google','Delhi'),

('Amazon','Hyderabad'),

('Microsoft','Bangalore'),

('IBM','Mumbai'),

('Accenture','Gurgaon');

company_name	city
Accenture	Gurgaon
Amazon	Hyderabad
Google	Delhi
HCL	Noida
IBM	Mumbai
Infosys	Bangalore
Microsoft	Bangalore
TCS	Mumbai
TechM	Pune
Wipro	Hyderabad
NULL	NULL

3. Works Table

CREATE TABLE Works (

employee_name VARCHAR(100), company_name VARCHAR(100), salary DECIMAL(10,2), PRIMARY KEY (employee_name, company_name),

FOREIGN KEY (employee_name) REFERENCES Employee(employee_name),

FOREIGN KEY (company_name) REFERENCES Company(company_name)

);

Insert values into Works

INSERT INTO Works VALUES

('Aman','TCS',50000),

('Ravi','Infosys',60000),

('Simran','Wipro',55000),

('Neha','HCL',65000),

('Raj','TechM',48000),

('Priya','Google',90000),

('Karan','Amazon',75000),

('Anjali','Microsoft',85000),

('Vikas','IBM',70000),

('Pooja','Accenture',62000);

employee_name	company_name	salary
Raj	TechM	48000.00
Aman	TCS	50000.00
Simran	Wipro	55000.00
Ravi	Infosys	60000.00
Pooja	Accenture	62000.00
Neha	HCL	65000.00
Vikas	IBM	70000.00
Karan	Amazon	75000.00
Anjali	Microsoft	85000.00
Priya	Google	90000.00
NULL	NULL	NULL

4. Manages Table

CREATE TABLE Manages (employee_name VARCHAR(100), manager_name VARCHAR(100),

PRIMARY KEY (employee_name),

FOREIGN KEY (employee_name) REFERENCES Employee(employee_name),

FOREIGN KEY (manager_name) REFERENCES Employee(employee_name)

);

Insert values into Manages

INSERT INTO Manages VALUES

('Aman','Ravi'),

('Simran','Neha'),

('Raj','Priya'),

('Karan','Anjali'),

('Vikas','Pooja'),

('Ravi','Simran'), ('Neha','Raj'),

('Priya','Karan'),

('Anjali','Vikas'),

('Pooja','Aman');

employee_name	manager_name
Pooja	Aman
Karan	Anjali
Priya	Karan
Simran	Neha
Vikas	Pooja
Raj	Priya
Neha	Raj
Aman	Ravi
Ravi	Simran
Anjali	Vikas
NULL	NULL

Q3. SQL Queries – Company Database

(a) Employees working in "First Bank Corporation" with salary > 10000

SELECT e.employee_name, e.street, e.city

FROM Employee e

```
JOIN Works w ON e.employee_name = w.employee_name

WHERE w.company_name = 'First Bank Corporation'

AND w.salary > 10000;
```

Result Grid | Filter Rows:

	employee_name	street	city

(b) Employees who do NOT work for "First Bank Corporation"

```
SELECT employee_name

FROM Employee

WHERE employee_name NOT IN (

    SELECT employee_name

    FROM Works

    WHERE company_name = 'First Bank Corporation'

);
```

	employee_name
▶	Aman
	Anjali
	Karan
	Neha
	Pooja
	Priya
	Raj
	Ravi
	Simran
	Vikas
•	NULL

(c) Employees earning more than EVERY employee of "Small Bank Corporation"

```
SELECT employee_name FROM Works

WHERE salary > ALL (

    SELECT salary

    FROM Works

    WHERE company_name = 'Small Bank Corporation'

);
```

	employee_name
▶	Raj
	Aman
	Simran
	Ravi
	Pooja
	Neha
	Vikas
	Karan
	Anjali
	Priya

(d) Companies located in ALL cities where "Small Bank Corporation" operates

```
 SELECT c1.company_name FROM Company c1

WHERE NOT EXISTS (

    SELECT city

    FROM Company

    WHERE company_name = 'Small Bank Corporation'

    AND city NOT IN (

        SELECT city
```

```
        FROM Company c2

        WHERE c2.company_name = c1.company_name

    )

);
```

Result Grid | Filter Ro

company_name
Accenture
Amazon
Google
HCL
IBM
Infosys
Microsoft
TCS
TechM
Wipro
NULL

(e) Company with MAXIMUM number of employees

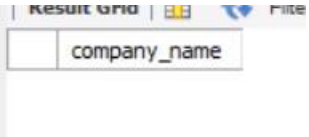
```
SELECT company_name

FROM Works

GROUP BY company_name

ORDER BY COUNT(employee_name) DESC

LIMIT 1;
```

company_name
Accenture

(f) Companies whose average salary > average salary of "First Bank Corporation"

```
SELECT company_name

FROM Works
```

```
GROUP BY company_name

HAVING AVG(salary) > (

    SELECT AVG(salary)

    FROM Works

    WHERE company_name = 'First Bank Corporation'

);
```



company_name

Q4. Table Creation – Bank Database

Branch Table

```
CREATE TABLE branch (     branch_name VARCHAR(50) PRIMARY KEY,     branch_city VARCHAR(50),     assets DECIMAL(12,2)

);


INSERT INTO branch VALUES

('SBI_Delhi', 'Delhi', 5000000),

('PNB_Mumbai', 'Mumbai', 7000000),

('HDFC_Brooklyn1', 'Brooklyn', 8000000),

('HDFC_Brooklyn2', 'Brooklyn', 6000000),

('ICICI_Chandigarh', 'Chandigarh', 5500000),

('Axis_Jalandhar', 'Jalandhar', 4500000),

('BOI_Harrison1', 'Harrison', 3000000),

('BOI_Harrison2', 'Harrison', 3500000),
```

('SBI_Amritsar', 'Amritsar', 4000000), ('PNB_Ludhiana', 'Ludhiana', 6500000);

branch_name	branch_city	assets
Axis_Jalandhar	Jalandhar	4500000.00
BOI_Harrison1	Harrison	3000000.00
BOI_Harrison2	Harrison	3500000.00
HDFC_Brooklyn1	Brooklyn	8000000.00
HDFC_Brooklyn2	Brooklyn	6000000.00
ICICI_Chandigarh	Chandigarh	5500000.00
PNB_Ludhiana	Ludhiana	6500000.00
PNB_Mumbai	Mumbai	7000000.00
SBI_Amritsar	Amritsar	4000000.00
SBI_Delhi	Delhi	5000000.00
NULL	NULL	NULL

Customer Table

CREATE TABLE customer (customer_name VARCHAR(50) PRIMARY KEY, customer_street VARCHAR(100), customer_city VARCHAR(50)

);

INSERT INTO customer VALUES

('Ravi', 'MainStreet', 'Brooklyn'),

('Priya', 'Street8', 'Delhi'),

('Arjun', 'Street9', 'Mumbai'),

('Ankit', 'Street3', 'Harrison'),

('Pooja', 'Street10', 'Chandigarh'),

('Varun', 'Street11', 'Jalandhar'), ('Ritika', 'Street12', 'Amritsar'),

('Manpreet', 'Street13', 'Ludhiana'),

('Sahil', 'Street14', 'Brooklyn'), ('Tina', 'Street15', 'Harrison');

customer_name	customer_street	customer_city
Ankit	Street3	Harrison
Arjun	Street9	Mumbai
Manpreet	Street13	Ludhiana
Pooja	Street10	Chandigarh
Priya	Street8	Delhi
Ravi	MainStreet	Brooklyn
Ritika	Street12	Amritsar
Sahil	Street14	Brooklyn
Tina	Street15	Harrison
Varun	Street11	Jalandhar
NULL	NULL	NULL

Loan Table

CREATE TABLE loan (loan_number INT PRIMARY KEY, branch_name VARCHAR(50), amount DECIMAL(10,2),

  FOREIGN KEY (branch_name) REFERENCES branch(branch_name)

);

INSERT INTO loan VALUES

(111, 'SBI_Delhi', 12000),

(112, 'PNB_Mumbai', 22000),

(113, 'HDFC_Brooklyn1', 18000),

(114, 'HDFC_Brooklyn2', 26000),

(115, 'ICICI_Chandigarh', 14000),

(116, 'Axis_Jalandhar', 17000),

(117, 'BOI_Harrison1', 21000),

(118, 'BOI_Harrison2', 23000),

(119, 'SBI_Amritsar', 31000), (120, 'PNB_Ludhiana', 29000);

loan_number	branch_name	amount
111	SBI_Delhi	12000.00
112	PNB_Mumbai	22000.00
113	HDFC_Brooklyn1	18000.00
114	HDFC_Brooklyn2	26000.00
115	ICICI_Chandigarh	14000.00
116	Axis_Jalandhar	17000.00
117	BOI_Harrison1	21000.00
118	BOI_Harrison2	23000.00
119	SBI_Amritsar	31000.00
120	PNB_Ludhiana	29000.00
NULL	NULL	NULL

Borrower Table

```
CREATE TABLE borrower (     customer_name VARCHAR(50),     loan_number INT,
    PRIMARY KEY (customer_name, loan_number),
    FOREIGN KEY (customer_name) REFERENCES customer(customer_name),
    FOREIGN KEY (loan_number) REFERENCES loan(loan_number)
);

INSERT INTO borrower VALUES
('Priya', 111),
('Arjun', 112),
('Ravi', 113),
('Sahil', 114),
('Pooja', 115),
('Varun', 116),
('Ankit', 117),
```

('Tina', 118),

('Ritika', 119),

('Manpreet', 120);

customer_name	loan_number
Priya	111
Arjun	112
Ravi	113
Sahil	114
Pooja	115
Varun	116
Ankit	117
Tina	118
Ritika	119
Manpreet	120
NULL	NULL

Account Table

CREATE TABLE account (account_number INT PRIMARY KEY, branch_name VARCHAR(50), balance DECIMAL(10,2),

  FOREIGN KEY (branch_name) REFERENCES branch(branch_name)

);

INSERT INTO account VALUES

(211, 'HDFC_Brooklyn1', 10000),

(212, 'HDFC_Brooklyn2', 11000),

(213, 'SBI_Delhi', 4000),

(214, 'PNB_Mumbai', 5000),

(215, 'BOI_Harrison1', 6000),

(216, 'BOI_Harrison2', 7000),

(217, 'ICICI_Chandigarh', 8000),

(218, 'Axis_Jalandhar', 9000),

(219, 'SBI_Amritsar', 3000), (220, 'PNB_Ludhiana', 2000);

account_number	branch_name	balance
211	HDFC_Brooklyn1	10000.00
212	HDFC_Brooklyn2	11000.00
213	SBI_Delhi	4000.00
214	PNB_Mumbai	5000.00
215	BOI_Harrison1	6000.00
216	BOI_Harrison2	7000.00
217	ICICI_Chandigarh	8000.00
218	Axis_Jalandhar	9000.00
219	SBI_Amritsar	3000.00
220	PNB_Ludhiana	2000.00
NULL	NULL	NULL

Depositor Table

CREATE TABLE depositor (customer_name VARCHAR(50), account_number INT,

PRIMARY KEY (customer_name, account_number),

FOREIGN KEY (customer_name) REFERENCES customer(customer_name), FOREIGN KEY (account_number) REFERENCES account(account_number)

);

INSERT INTO depositor VALUES

('Ravi', 211),

('Ravi', 212),

('Priya', 213),

('Arjun', 214),

('Ankit', 215),

('Tina', 216),

('Pooja', 217),

('Varun', 218),

('Ritika', 219),

('Manpreet', 220);

customer_name	account_number
Ravi	211
Ravi	212
Priya	213
Arjun	214
Ankit	215
Tina	216
Pooja	217
Varun	218
Ritika	219
Manpreet	220
NULL	NULL

Q5. SQL Queries – Bank Database

a) Customers who have an account but no loan

```
SELECT customer_name

FROM depositor

WHERE customer_name NOT IN (

    SELECT customer_name FROM borrower

);
```

Result Grid | Filter Rows:

customer_name

b) Customers living on same street & city as "Smith"

```
SELECT customer_name

FROM customer

WHERE (customer_street, customer_city) IN (

    SELECT customer_street, customer_city

    FROM customer

    WHERE customer_name = 'Smith'

);
```

Result Grid | Filter R

customer_name
NULL

c) Branch names where customers have accounts & live in "Harrison"

```
SELECT DISTINCT b.branch_name

FROM branch b

JOIN account a ON b.branch_name = a.branch_name

JOIN depositor d ON a.account_number = d.account_number

JOIN customer c ON d.customer_name = c.customer_name

WHERE c.customer_city = 'Harrison';
```

Result Grid

branch_name
BOI_Harrison1
BOI_Harrison2

d) Customers who have accounts in all branches in "Brooklyn"

```
SELECT d.customer_name

FROM depositor d

JOIN account a ON d.account_number = a.account_number

JOIN branch b ON a.branch_name = b.branch_name

WHERE b.branch_city = 'Brooklyn'

GROUP BY d.customer_name

HAVING COUNT(DISTINCT b.branch_name) = (

    SELECT COUNT(*) FROM branch WHERE branch_city = 'Brooklyn'

);
```

Result Grid

customer_name
Ravi

e) Total sum of all loan amounts

```
SELECT SUM(amount) AS total_loan_amount

FROM loan;
```

Result Grid

total_loan_amount
213000.00

f) Branches with assets greater than at least one branch in "Brooklyn"

```
SELECT branch_name
```

FROM branch

WHERE assets > ANY (

SELECT assets FROM branch WHERE branch_city = 'Brooklyn');

branch_name
HDFC_Brooklyn1
PNB_Ludhiana
PNB_Mumbai
NULL

Q6. Constraints and Relationships

a) Define appropriate foreign keys for the following schema:

STUDENT, COURSE, ENROLL, BOOK_ADOPTION, TEXT

ENROLL Table student_id → references STUDENT(student_id) course_id → references COURSE(course_id)

BOOK_ADOPTION Table course_id → references COURSE(course_id) book_id → references TEXT(book_id)

b) Implement all constraints using MySQL:

PRIMARY KEY FOREIGN KEY NOT NULL UNIQUE CHECK

STUDENT TABLE

CREATE TABLE students (student_id INT PRIMARY KEY, name VARCHAR(50) NOT NULL, email VARCHAR(100) UNIQUE, age INT CHECK (age >= 18)

);

COURSE TABLE

CREATE TABLE courses (course_id INT PRIMARY KEY, course_name VARCHAR(100) NOT NULL, credits INT CHECK (credits > 0)

);

TEXT TABLE

CREATE TABLE text (book_id INT PRIMARY KEY, title VARCHAR(100) NOT NULL, publisher VARCHAR(100), price DECIMAL(8,2) CHECK (price > 0)

);

ENROLL TABLE

CREATE TABLE enroll (student_id INT, course_id INT, enroll_date DATE NOT NULL,

PRIMARY KEY (student_id, course_id),

FOREIGN KEY (student_id) REFERENCES students(student_id),

FOREIGN KEY (course_id) REFERENCES courses(course_id)

);

BOOK_ADOPTION TABLE

CREATE TABLE book_adoption (course_id INT, book_id INT, semester VARCHAR(20) NOT NULL,

PRIMARY KEY (course_id, book_id),

FOREIGN KEY (course_id) REFERENCES courses(course_id),

```
    FOREIGN KEY (book_id) REFERENCES text(book_id)
);
```

Q7. Advanced Tasks

a)Create a VIEW for employees working in First Bank Corporation.

```
CREATE VIEW first_bank_employees AS
SELECT * FROM employee
WHERE company_name = 'First Bank Corporation';
```

b)Create an INDEX on salary in the works table.

```
CREATE INDEX idx_salary
ON works(salary);
```

(c)Write a query using GROUP BY and HAVING.

```
SELECT company_name, AVG(salary) AS avg_salary
FROM works
GROUP BY company_name
HAVING AVG(salary) > 50000;
```

Result Grid | Filter Rows:

company_name	avg_salary
Accenture	62000.000000
Amazon	75000.000000
Google	90000.000000
HCL	65000.000000
IBM	70000.000000
Infosys	60000.000000
Microsoft	85000.000000
Wipro	55000.000000

Assignment-7

Q-1 You are managing a bank database.

Situation:

- **Table: Accounts(account_id, name, balance)**
- **User: clerk1**

Tasks:

b. **Transfer ₹5000 from account A to account B.**

c. **If any error occurs, undo the transaction.**

d. **If everything is correct, save changes permanently.**

Write SQL commands for the above.

```
CREATE TABLE Accounts (     account_id INT PRIMARY KEY,     name VARCHAR(50),
   balance INT
);

INSERT INTO Accounts VALUES (1, 'roshan', 20000);
INSERT INTO Accounts VALUES (2, 'virag', 15000);
INSERT INTO Accounts VALUES (3, 'amrinder', 30000);
INSERT INTO Accounts VALUES (4, 'pratik', 18000);
INSERT INTO Accounts VALUES (5, 'mridul', 25000);
```

INSERT INTO Accounts VALUES (6, 'Uttam', 23000);

account_id	name	balance
1	roshan	10000
2	virag	20000
3	amrinder	30000
4	pratik	18000
5	mridul	25000
NULL	NULL	NULL

a. Give clerk1 permission to view and update account details.

CREATE USER 'clerk1'@'localhost' IDENTIFIED BY 'password123';

GRANT SELECT, UPDATE ON Accounts TO 'clerk1'@'localhost';

b. Transfer ₹5000 from account A to account B.

START TRANSACTION;

UPDATE Accounts

SET balance = balance - 5000

WHERE name = 'roshan';

UPDATE Accounts

SET balance = balance + 5000

WHERE name = 'virag';

c. **If any error occurs, undo the transaction.**

SAVEPOINT transfer_done;

d. **If everything is correct, save changes permanently**

COMMIT;

account_id	name	balance
1	roshan	5000
2	virag	25000
3	amrinder	30000
4	pratik	18000
5	mridul	25000
NULL	NULL	NULL

Q-2 You are both DBA + Developer.

Situation:

- **Table: Projects(id, name, budget)**
- **User: manager1**

Write a full SQL script.

CREATE TABLE Projects (id INT PRIMARY KEY, name VARCHAR(50), budget INT

);

INSERT INTO Projects VALUES (1, 'roshan', 80000);

INSERT INTO Projects VALUES (2, 'virag', 60000);

INSERT INTO Projects VALUES (3, 'amrinder', 120000);

INSERT INTO Projects VALUES (4, 'pratik', 50000);

INSERT INTO Projects VALUES (5, 'mridul', 90000);

	id	name	budget
▶	1	roshan	80000
	2	virag	60000
	3	amrinder	120000
	4	pratik	50000
	5	mridul	90000

CREATE USER 'manager1'@'localhost' IDENTIFIED BY 'pass123';

a. Grant ALL privileges to manager1.

GRANT ALL PRIVILEGES ON Projects TO 'manager1'@'localhost';

FLUSH PRIVILEGES;

START TRANSACTION;

b. Insert 2 records.

-- First insert

INSERT INTO Projects VALUES (6, 'AI System', 100000);

-- Second insert

INSERT INTO Projects VALUES (7, 'Web App', 50000);

id	name	budget
1	roshan	80000
2	virag	60000
3	amrinder	120000
4	pratik	50000
5	mridul	90000
6	AI System	100000
7	Web App	50000
NULL	NULL	NULL

c. Create SAVEPOINT after first insert.

SAVEPOINT sp1;

d. Delete second record and rollback only that deletion.

DELETE FROM Projects WHERE id = 7;

ROLLBACK TO sp1;

SELECT * FROM Projects;

id	name	budget
1	roshan	80000
2	virag	60000
3	amrinder	120000
4	pratik	50000
5	mridul	90000
6	AI System	100000

e. Revoke DELETE permission from manager1.

REVOKE DELETE ON Projects FROM 'manager1'@'localhost';

FLUSH PRIVILEGES;

f. Commit final state.

COMMIT;

Assignment-8

Objective:To develop a foundational understanding of PL/SQL programming concepts, including declaration and usage of variables, implementation of control structures, and construction of basic SQL queries within PL/SQL blocks.

PL/SQL Programming Concepts

Introduction to PL/SQL

PL/SQL (Procedural Language/Structured Query Language) is Oracle's extension of SQL that combines SQL statements with procedural programming features such as variables, loops, and conditional statements. It allows developers to write complex logic that executes directly inside the database.

Variables in PL/SQL

Definition

A variable in PL/SQL is a named memory location used to store data values temporarily during program execution. Variables must be declared before use in the DECLARE section.

Key Points

- Defined with a datatype (NUMBER, VARCHAR2, etc.)
- Can store query results
- Used for calculations and logic

Example

```
SET SERVEROUTPUT ON;
```

```
DECLARE

  v_name VARCHAR2(50); BEGIN

  v_name := 'Amrinder';

  DBMS_OUTPUT.PUT_LINE('Name: ' || v_name); END;

/
```

```
Name: Amrinder

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.005
```

Control Structures in PL/SQL

Definition

Control structures are used to control the flow of execution in a PL/SQL program. They allow decision-making and repetition of tasks.

Types

1. Conditional Statements (IF-ELSE)
2. Looping Statements (FOR, WHILE)

IF-ELSE Statement

Definition

Used to execute code based on a condition.

Example

```
SET SERVEROUTPUT ON;


DECLARE
```

```
  v_num NUMBER := 10; BEGIN

  IF v_num > 5 THEN

    DBMS_OUTPUT.PUT_LINE('Number is greater than 5');

  ELSE

    DBMS_OUTPUT.PUT_LINE('Number is less than or equal to 5');    END IF;

END;
/
```

```
Number is greater than 5

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.010
```

LOOP (FOR LOOP)

Definition

Used to execute a block of code multiple times.

Example

```
SET SERVEROUTPUT ON;



BEGIN

  FOR i IN 1..5 LOOP

    DBMS_OUTPUT.PUT_LINE('Value: ' || i);

  END LOOP;
```

END;
/

```
Value: 1
Value: 2
Value: 3
Value: 4
Value: 5


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.004
```

SQL Queries in PL/SQL

Definition

PL/SQL allows embedding SQL queries within procedural code. Data retrieved from SQL queries can be stored into variables using the SELECT INTO statement. **Key Points**

- Used inside BEGIN...END
- Must return a single row (for SELECT INTO)
- Integrates SQL with logic

Example

```
SET SERVEROUTPUT ON;



DECLARE

  v_salary NUMBER;

BEGIN

  SELECT salary INTO v_salary

  FROM hr.employees

  WHERE employee_id = 100;
```

```
  DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);

END;

/
```

```
Salary: 24000

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.013
```

Assignment-9

Objective:Writing PL/SQL code with CASE statements, loops, and exception handling mechanisms for effective database programming.

PL/SQL with CASE Statements, Loops, and Exception Handling

1. CASE Statement in PL/SQL

Definition

A CASE statement is used to perform conditional logic by comparing a variable or expression against multiple values. It improves readability over multiple IF-ELSE conditions. **Types**

- Simple CASE (compares one value)
- Searched CASE (uses conditions)

Example (Simple CASE)

```
DECLARE

  v_marks NUMBER := 85;    v_grade VARCHAR2(10); BEGIN

  v_grade :=

    CASE

      WHEN v_marks >= 90 THEN 'A'
```

```
      WHEN v_marks >= 75 THEN 'B'
      WHEN v_marks >= 50 THEN 'C'
      ELSE 'Fail'
    END;

  DBMS_OUTPUT.PUT_LINE('Grade: ' || v_grade);
END;
/
```

```
Grade: B

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.007
```

---

2. Loops in PL/SQL

Definition

Loops are used to execute a block of code repeatedly until a condition is met. **Types**

- FOR LOOP
- WHILE LOOP
- SIMPLE LOOP

Example (FOR LOOP)

```
BEGIN
  FOR i IN 1..5 LOOP
    DBMS_OUTPUT.PUT_LINE('Iteration: ' || i);
  END LOOP;
```

END;
/

```
Iteration: 1
Iteration: 2
Iteration: 3
Iteration: 4
Iteration: 5


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.006
```

Example (WHILE LOOP)

```
DECLARE
  i NUMBER := 1;
BEGIN
  WHILE i <= 5 LOOP
    DBMS_OUTPUT.PUT_LINE('Value: ' || i);       i := i + 1;
  END LOOP;
END;
/
```

```
Value: 1
Value: 2
Value: 3
Value: 4
Value: 5


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.007
```

---

3. Exception Handling in PL/SQL

Definition

Exception handling is used to manage runtime errors in a PL/SQL block. It prevents program failure and allows graceful handling of errors. **Common Exceptions**

- NO_DATA_FOUND
- TOO_MANY_ROWS
- ZERO_DIVIDE

Example

```
DECLARE
  v_salary NUMBER;
BEGIN
  SELECT salary INTO v_salary
  FROM hr.employees
  WHERE employee_id = 9999;  -- likely no such employee

  DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No employee found');

  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Some error occurred');
END;
/
```

```
No employee found

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.013
```

---

4. Combined Example

This shows CASE + LOOP + EXCEPTION together.

```
DECLARE
  v_marks NUMBER := 78;    v_grade VARCHAR2(10); BEGIN
  -- CASE statement    v_grade :=
    CASE
      WHEN v_marks >= 90 THEN 'A'
      WHEN v_marks >= 75 THEN 'B'
      WHEN v_marks >= 50 THEN 'C'
      ELSE 'Fail'
    END;


  DBMS_OUTPUT.PUT_LINE('Grade: ' || v_grade);


  -- LOOP
  FOR i IN 1..3 LOOP
    DBMS_OUTPUT.PUT_LINE('Loop count: ' || i);
  END LOOP;
```

```
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error occurred');
END;
/
```

```
Grade: B
Loop count: 1
Loop count: 2
Loop count: 3


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.003
```